Module 8 — Messaging (Kafka & RabbitMQ)

> Interviewers probe delivery guarantees (at-least/at-most/exactly-once), ordering, consumer groups, offsets, retries, and DLQs. Know **when Kafka vs RabbitMQ** and how each achieves ordering and reliability.

8.1 Kafka vs RabbitMQ — the fundamental difference

	Kafka	RabbitMQ
Model	distributed **commit log**	traditional **message broker/queue**
Consumption	pull; offset-based; **messages retained**	push; **removed on ack**
Ordering	per-partition	per-queue
Replay	yes (re-read offsets)	no (once acked, gone)
Throughput	very high (millions/s)	high, lower than Kafka
Routing	topic/partition	exchanges (direct/topic/fanout/headers)
Best for	event streaming, logs, high volume, replay, CQRS/ES	task queues, complex routing, RPC, per-message TTL/priority

One-liner: Kafka is a durable, replayable *log* optimized for throughput and streaming; RabbitMQ is a flexible *broker* optimized for routing and per-message delivery semantics.

8.2 Kafka core concepts

Producer / Topic / Partition / Offset

- **Topic** — named stream, split into **partitions** (unit of parallelism & ordering).
- **Partition** — ordered, append-only log; each message has a monotonically increasing **offset**.
- **Producer** — writes to a partition; the **partitioner** picks the partition (by `key` hash, or round-robin if no key). **Same key → same partition → ordering.**
- **Offset** — position of a message in a partition; consumers track their offset.

Consumer & Consumer Groups

- A **consumer group** shares the work: each partition is consumed by **exactly one** consumer in the group → parallelism = min(#partitions, #consumers).
- Different groups each get **all** messages (pub/sub fan-out).
- **Rebalancing** reassigns partitions when consumers join/leave.
- **Committed offset** marks progress; on restart the group resumes from it.

ASCII — Topic / Partitions / Group

```
Topic "orders" (3 partitions)
  P0: [o0][o3][o6]  <- consumer A (group G)
  P1: [o1][o4][o7]  <- consumer B (group G)
  P2: [o2][o5][o8]  <- consumer C (group G)
key(customerId) -> hash -> fixed partition -> per-customer ordering
group H reads ALL partitions independently (fan-out)
```

Ordering

Kafka guarantees ordering **only within a partition**. For per-entity ordering, key by that entity id. Global ordering ⇒ single partition (kills parallelism). Beware: `max.in.flight.requests > 1` + retries can reorder unless `enable.idempotence=true`.

Interview Q

- What determines the partition a message goes to?
- How do consumer groups provide scaling and fan-out?
- How does Kafka guarantee ordering, and its limits?
- What triggers a rebalance; why can it cause pauses/duplicates?

8.3 Delivery Guarantees: At-most / At-least / Exactly-once

Guarantee	Behavior	How
At-most-once	may lose, never duplicate	commit offset **before** processing / fire-and-forget
At-least-once	never lose, may duplicate	commit offset **after** processing (default; needs idempotent consumer)
Exactly-once	no loss, no dup	Kafka idempotent producer + **transactions** (read-process-write) or idempotent consumer + dedup

Kafka exactly-once (EOS)

`enable.idempotence=true` (dedup producer retries) + transactional producer (`transactional.id`, `initTransaction/commitTransaction`) + consumer `isolation.level=read_committed`. True EOS holds **within Kafka** (topic→topic). When a side effect hits an external DB, you still need **idempotency** (Module 7).

Producer acks

- `acks=0` (fire-and-forget, may lose), `acks=1` (leader only), `acks=all` + `min.insync.replicas` (durable, no loss on single-broker failure).

ASCII — commit timing decides the guarantee

```
at-least-once:  poll -> PROCESS -> commit offset   (crash before commit -> reprocess)
at-most-once:   poll -> commit offset -> PROCESS    (crash after commit -> lost)
```

Interview Q / Follow-ups

- Difference between the three guarantees and how to configure each.
- Is exactly-once “real”? *(within Kafka via txns; end-to-end needs idempotency.)*
- What do `acks=all` + `min.insync.replicas` protect against?
- Why is at-least-once the pragmatic default?

8.4 Retry & Dead Letter Queue (DLQ)

Core Concept

Transient failures → **retry with backoff**; poison messages (always fail) → route to a **Dead Letter Queue/Topic** after N attempts so the pipeline isn’t blocked. In Spring Kafka: `DefaultErrorHandler` + `DeadLetterPublishingRecoverer` (retryable vs non-retryable exceptions, backoff). RabbitMQ: DLX (dead-letter exchange) + TTL for delayed retry.

ASCII

```
consume -> fail -> retry (backoff) x N -> still failing -> DLQ (orders.DLT)
DLQ monitored/alerted -> manual or automated reprocessing
```

Common Mistakes / Best Practices

Blocking retries on the main consumer (stalls the partition) — prefer non-blocking retry topics; infinite retries on poison messages; no DLQ monitoring. Always attach metadata (original topic, exception, attempts) to DLQ records.

Interview Q

What is a DLQ; when does a message go there; blocking vs non-blocking retries; how to reprocess a DLQ.

8.5 Consumer Lag & Offsets in Practice

- **Consumer lag** = latest offset − committed offset (how far behind). High lag = consumers can't keep up → scale consumers (up to #partitions), optimize processing, or increase partitions.
- **Offset commit**: auto (`enable.auto.commit`, risky) vs manual (`ack-mode=MANUAL`, precise). Commit after successful processing for at-least-once.

Interview Q

How to diagnose Kafka lag; how to scale consumers; auto vs manual commit.

8.6 RabbitMQ concepts (contrast)

- **Exchange** types: **direct** (routing key exact), **topic** (wildcard patterns), **fanout** (broadcast), **headers**.
- **Queue** bound to exchange; consumers ack (`basic.ack`/nack); unacked messages redelivered.
- **Prefetch (QoS)** limits unacked messages per consumer (backpressure).
- **DLX** + message TTL for retry/delay; priority queues.

Interview Q

Exchange types and routing; ack/nack/requeue; prefetch and why it matters.

Module 8 — One-Page Cheat Sheet

Topic	Key point
Kafka vs Rabbit	log+replay+throughput vs broker+routing+per-msg semantics
Partition	ordering + parallelism unit; key → same partition
Consumer group	1 partition ↔ 1 consumer in group; other groups fan-out
Offset	consumer progress; commit timing sets guarantee
At-least-once	commit after process; needs idempotent consumer (default)
At-most-once	commit before process; may lose
Exactly-once	idempotent+transactional producer; end-to-end needs idempotency
acks=all + ISR	durability, no loss on broker failure
DLQ	poison messages after N retries; monitor + reprocess
Lag	latest−committed; scale up to #partitions
Rabbit exchanges	direct/topic/fanout/headers; prefetch backpressure

Module 8 — Top Interview Questions

1. Kafka vs RabbitMQ — when each?
2. How does Kafka guarantee ordering; its limits?
3. Explain consumer groups, partitions, offsets.
4. At-most vs at-least vs exactly-once; how to configure.
5. Is exactly-once real end-to-end? (idempotency)
6. What is a DLQ; blocking vs non-blocking retry.
7. `acks` and `min.insync.replicas`.
8. How to diagnose and fix consumer lag.
9. Auto vs manual offset commit.
10. RabbitMQ exchange types and routing.

Module 8 — Common Mistakes

- Assuming global ordering across partitions.
- Auto-commit causing lost/duplicate processing.
- No idempotency with at-least-once delivery.
- Blocking retries stalling a partition; no DLQ.
- More consumers than partitions (idle consumers).

Module 8 — Mock Interview

1. *“Guarantee per-customer event ordering at scale.”* → key by customerId → same partition; don’t rely on global order.
2. *“A consumer keeps crashing on one bad message and blocks everything.”* → route to DLQ after N retries (non-blocking retry topics).
3. *“How do you avoid double-charging on Kafka retries?”* → at-least-once + idempotent consumer (dedup on eventId), and/or transactional EOS.
4. *“Consumer lag is growing.”* → scale consumers up to #partitions, add partitions, speed up processing, batch.

5. *“Choose Kafka or RabbitMQ for a task queue with priorities and per-message TTL.”* → RabbitMQ (priority queues, TTL, flexible routing).

Next → Module 9: Redis.